PI AGENT

Hooks, Events, Commands and Handlers

A compact study guide to the internal vocabulary and lifecycle of the Pi coding-agent harness.

"A person can outsource intelligence but not understanding."

The central mental model:

Action or command -> event -> pi.on() -> handler

Prepared 24 July 2026

1. What Pi is

Pi is best described as a minimal, extensible agent harness. It connects an LLM to an agent loop, tools, session history, terminal interaction and extensions. Its SDK and lower-level packages can also be used as an agent framework or toolkit.

Official Pi website: https://pi.dev/

2. The four terms that matter

Term	Meaning	Simple reading
`Action / command`	Something the user, LLM or Pi initiates.	"Do this."
`Event`	A notification that something has happened or is about to happen.	"This happened."
`pi.on()`	The API used to subscribe a function to an event.	"When this happens..."
`Handler`	The function that runs when the subscribed event occurs.	"...run this code."

The complete relationship

Action / command -> Event -> pi.on() -> Handler

Important precision: Pi exposes one event-subscription method, `pi.on()`. In the current extension API it accepts 33 named events. People often call each subscription a hook, so the words overlap in ordinary discussion.

3. Reading pi.on() correctly

```
pi.on("tool_call", handler)
```

Read this as: "Whenever the `tool_call` event occurs, run `handler`."

`if` checks a condition when execution reaches it. `pi.on()` registers a listener that remains available for future occurrences of an event.

4. The /compact example

`/compact` is a slash command. It is neither an event nor a hook. It initiates work that causes events.



An extension may attach a handler before compaction. That handler can observe, cancel or customize the operation. If compaction proceeds, Pi later emits the completion event.

5. One ordinary prompt produces many events

Events are not limited to typed commands. Most are generated automatically while Pi performs its internal work. A simplified prompt lifecycle is:

```
User prompt -> input -> before_agent_start -> agent_start -> turn_start -> context -> provider request and response events -> message events -> optional tool events -> turn_end -> agent_end -> agent_settled
```

Turn versus agent run: one agent run may contain several turns. If the LLM calls a tool, Pi executes the tool, returns its result to the LLM, and another turn begins.

6. Three useful distinctions

Before events occur before an operation and may allow an extension to influence it. Examples include `session_before_compact` and `tool_call`.

After events report what has completed. Examples include `session_compact` and `tool_result`.

Lifecycle events describe Pi's continuing internal process. They do not require a specific slash command. Examples include `turn_start`, `message_update` and `agent_settled`.

7. Complete reference: all 33 hookable events

The table answers: What activity causes each event? These are available connection points; they do not mean 33 handlers are always running.

#	Area	Event	What causes it
1	Startup	`project_trust`	Pi enters a project whose trust has not been decided.
2	Startup	`resources_discover`	Startup or reload searches for skills, prompts and themes.
3	Session	`session_start`	Startup, reload, /new, /resume, /fork or /clone.
4	Session	`session_info_changed`	/name, RPC or pi.setSessionName() changes the session name.
5	Session	`session_before_switch`	/new or /resume is requested.
6	Session	`session_before_fork`	/fork or /clone is requested.
7	Session	`session_before_compact`	/compact or automatic compaction starts.
8	Session	`session_compact`	Compaction finishes successfully.
9	Session	`session_shutdown`	Exit, reload, session switch, new session or fork.
10	Session	`session_before_tree`	/tree navigation is requested.
11	Session	`session_tree`	Tree navigation finishes.
12	LLM	`context`	Pi prepares messages before every LLM call.
13	Provider	`before_provider_request`	The provider payload has been built and is about to be sent.
14	Provider	`before_provider_headers`	The outgoing HTTP request headers have been assembled.
15	Provider	`after_provider_response`	The model provider returns an HTTP response.
16	Agent	`before_agent_start`	An accepted prompt is about to enter the agent loop.
17	Agent	`agent_start`	A low-level agent run begins.
18	Agent	`agent_end`	That low-level agent run ends.
19	Agent	`agent_settled`	No automatic retry, compaction or follow-up remains.
20	Turn	`turn_start`	A new LLM-and-tools turn begins.
21	Turn	`turn_end`	That turn finishes.
22	Message	`message_start`	A user, assistant or tool-result message begins.
23	Message	`message_update`	Assistant output arrives while streaming.
24	Message	`message_end`	A message is finalized.
25	Tool	`tool_execution_start`	Pi begins processing a tool request.
26	Tool	`tool_execution_update`	A running tool provides partial output.
27	Tool	`tool_execution_end`	Tool processing is finalized.
28	Model	`model_select`	/model, model cycling, restoration or pi.setModel().
29	Model	`thinking_level_select`	Thinking control, model change or pi.setThinkingLevel().
30	Tool	`tool_call`	The LLM requests a tool, immediately before execution.
31	Tool	`tool_result`	The tool finishes and returns its result.
32	Shell	`user_bash`	The user executes a command using ! or !!.
33	Input	`input`	Pi receives user, RPC or extension-generated input.

8. How to study an unfamiliar Pi extension

Do not try to understand the entire extension at once. Read each subscription as a sentence:

```
1. Find pi.on(...). 2. Identify the event name. 3. Ask what action or internal step produces it. 4. Read the handler to see what it observes, blocks or changes. 5. Follow only the helper functions called by that handler.
```

For example, when you see `pi.on("tool_call", ...)`, first translate it to: "Before a requested tool executes, run this code." Only then inspect the handler's details.

9. Compact memory aid

Command: asks Pi to do something.
Event: announces where Pi is in the process.
pi.on(): connects your extension to that announcement.
Handler: contains the extension's reaction.

10. Official links for continued study

- Pi website - https://pi.dev/
- Pi source repository - github.com/earendil-works/pi
- Official extension documentation and lifecycle
- Current ExtensionAPI event type definitions
- Lower-level pi-agent-core package

Version note: this guide reflects Pi's public documentation and source definitions checked on 24 July 2026. Pi evolves quickly, so the official ExtensionAPI type definition is the final authority if names or behavior change.